Lecture 6

Introduction to Middleware Architectures and Technologies

Middleware Architectures and Technologies

Outlines

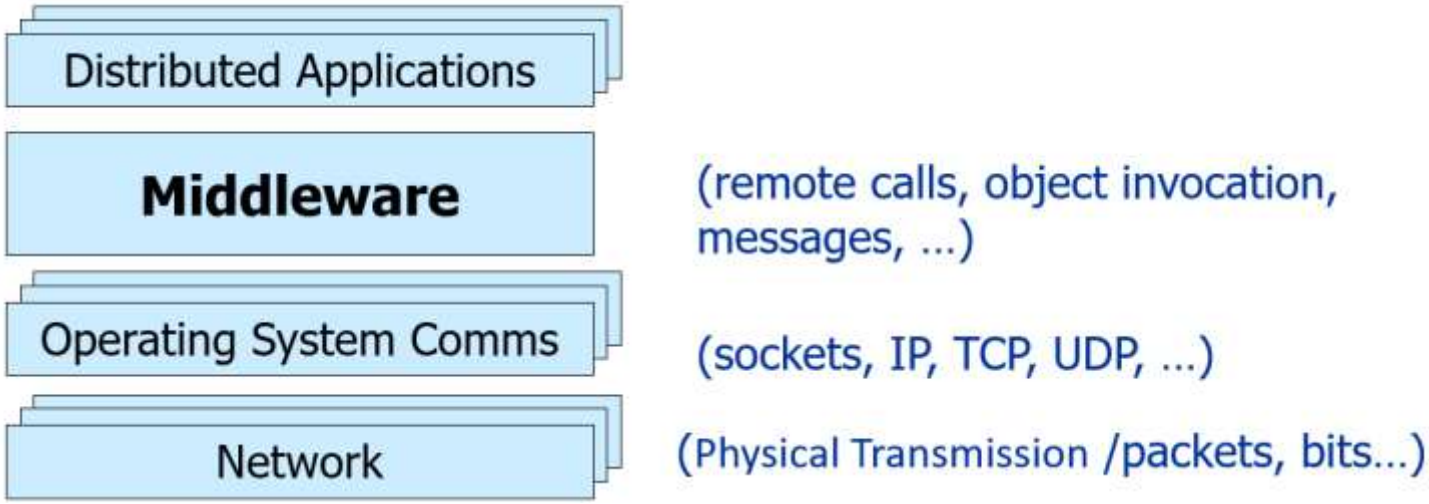
- Introduction to Middleware
- Remote Procedure Call (RPC)
- Object-Oriented Middleware (OOM)
- Message-Oriented Middleware (MOM)
- Event-Based Middleware

Introduction to Middleware

Middleware is a software layer that:

- Resides between the operating system and distributed applications
- Hides complexity and heterogeneity of distributed systems
- Acts as a bridges low-level OS communications and programming language abstractions
- Provides common programming abstraction and infrastructure for distributed apps

Middleware Architecture:



Introduction to Middleware

Middleware provides support for a variety of critical aspects in distributed computing:

- Naming, location, and service discovery
- Replication
- Protocol handling and management of communication faults
- Quality of Service (QoS)
- Synchronization, concurrency control, and transaction management
- Storage mechanisms
- Access control and authentication

Introduction to Middleware

Middleware solutions can be categorized along several dimensions:

- Request/Reply versus Asynchronous Messaging
- Language-specific versus Language-independent
- Proprietary versus Standards-based
- Small-scale versus Large-scale
- Tightly-coupled versus Loosely-coupled components

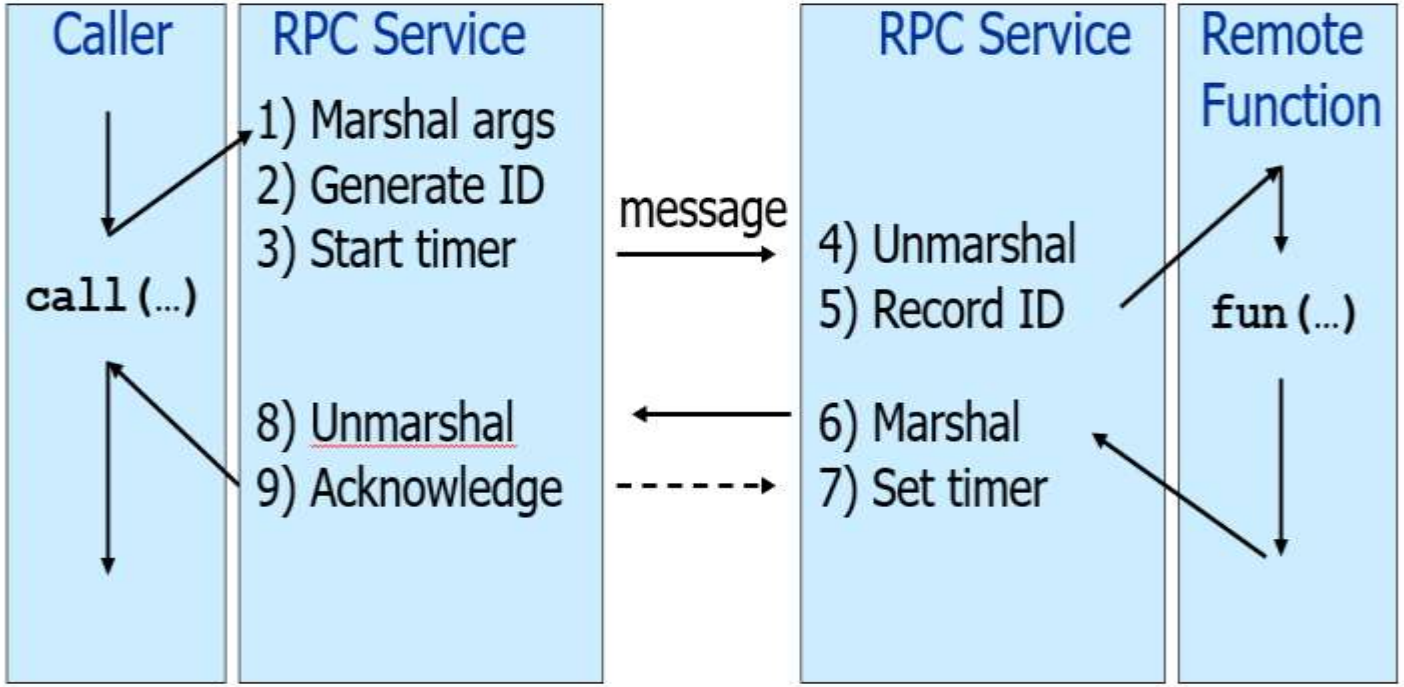
Remote Procedure Call (RPC)

- RPC is a mechanism that masks remote function calls, making them appear as local function/procedure calls to the programmer.
- It operates on a client/server model and employs a request/reply paradigm, typically implemented using message passing.
- RPC involves the marshalling (packaging) of function parameters and return values on the client side and unmarshalling on the server side.

Remote Procedure Call (RPC)

RPC Execution Flow

1. Client makes local procedure call
2. Client stub marshals parameters
3. RPC runtime sends message to server
4. Server stub unmarshals parameters
5. Server executes procedure
6. Return value follows reverse path



RPC Properties

- RPC provides a familiar Language-level pattern of **function call**
  - Easy to understand for programmer
- It utilizes **Synchronous request/reply** interaction
  - Aligns /natural from a programming language point-of-view
  - Matches replies to requests
  - Built in synchronisation of requests and replies
- RPC aims to achieve **Distribution transparency** (in the no-failure case)
  - Hides the complexity of a distributed system
- RPC systems offer varying degrees of **reliability** guarantees
  - Handle potential failures in distributed environments

RPC Failure Modes

- RPC systems must address failure scenarios, including network congestion, server overload, and client/server failures.
- Common invocation semantics include:

Exactly once (RPC system retries a few times)

- The RPC system retries the call a few times to attempt execution, but it may still return an error. It's important to note that "exactly once" cannot be strictly guaranteed.

Maybe or at most once (RPC system tries once)

- The RPC system tries the call only once, and the programmer is responsible for handling any potential errors and retries.

Advantages and Disadvantages

Advantages

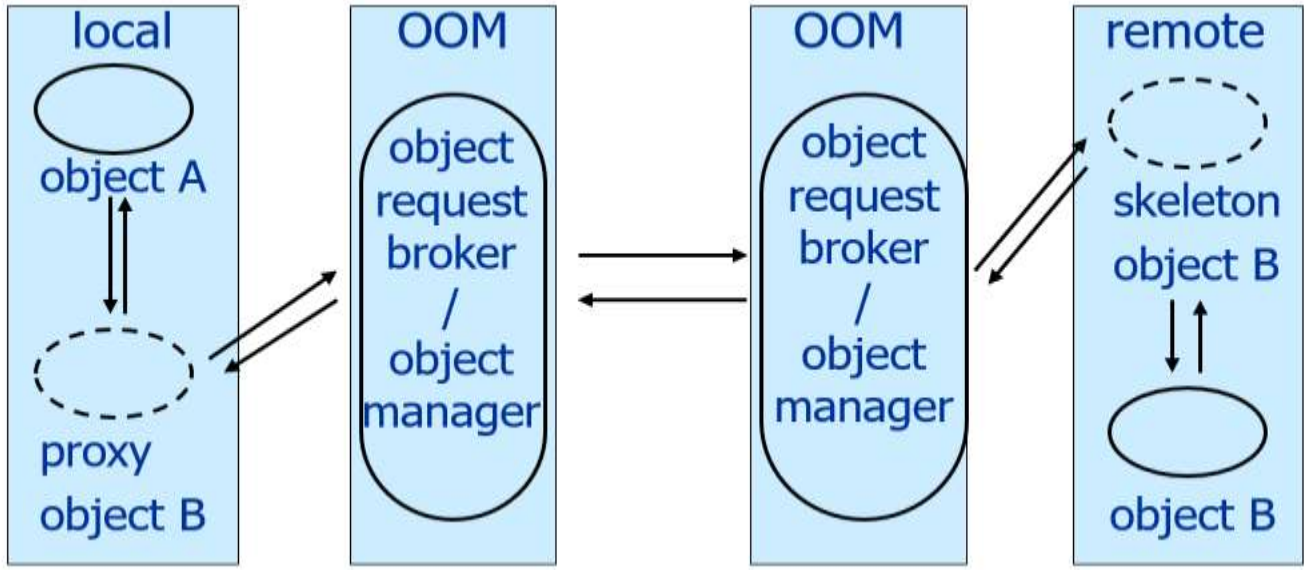
- Simple programming model
- Natural synchronization
- Handles some failures automatically

Disadvantages

- Synchronous nature of request/reply in RPC can lead to tight coupling between clients and servers.
  - This can cause client blocking if the server is slow or unresponsive, potentially requiring multi-threaded programming on the client-side.
  - slow or failed clients can delay servers during the reply phase, making multi-threading essential on the server-side as well.
- Distribution transparency is not always feasible;
  - some distributed system issues cannot be fully masked.
- RPC paradigm is function-oriented rather than object-oriented,
  - it involves invoking functions on servers, not methods on objects.

Object-Oriented Middleware (OOM)

- **Objects** can be *local* or *remote*
- **Object references** can be *local* or *remote*
- Remote objects have visible **remote interfaces**
- Remote objects as being local object using **proxy objects**
- Supports remote method invocation



OOM Properties

- Support for object-oriented programming model
  - objects, methods, interfaces, encapsulation, …
  - exceptions (were also in some RPC systems e.g. Mayflower)
- Like RPC, OOM typically uses synchronous request/reply interaction
- OOM provides location transparency
  - The middleware (Object Request Broker or ORB) handles the mapping of object references to their physical locations.
- Construction of services in multiple servers are easier to build with OOM
  - It allows programming in terms of object interfaces rather than server addresses

Examples of OOM

Java Remote Method Invocation (RMI)

- RMI is a Java-specific OOM solution for distributed objects in Java.
- It uses RMI compilers to generate proxies and skeletons, which facilitate communication between remote objects.
- The RMI registry is used for looking up remote object interfaces.
- RMI is a single-language system, meaning it is designed for Java-to-Java communication, entire system written in Java

```
public interface PrintService extends Remote {
  int print(Vector printJob) throws RemoteException;
}
```

Examples of OOM

Common Object Request Broker Architecture (CORBA)

- CORBA is an open standard developed by the OMG (Object Management Group) and is designed to be language- and platform-independent.

Key components of CORBA include:

- Object Request Broker (ORB): The core middleware component that enables communication between objects.
- General Inter-ORB Protocol (GIOP): The standard protocol for communication between ORBs.
- Interoperable Object References (IOR): References that contain the location of remote objects.
- Interface Definition Language (IDL): A language used to define remote interfaces in a language-neutral way. IDL compilers generate stubs (proxies) and skeletons.

- CORBA supports dynamic remote method invocation
- CORBA provides an Interface Repository for querying existing remote interfaces
- CORBA provides an Implementation Repository for activating remote objects on demand.

Examples of OOM

CORBA IDL Details

- CORBA IDL is used to define language-independent remote interfaces,
  - Allowing for mappings to various programming languages like C++, Java, and Smalltalk.
  - The IDL compiler translates IDL definitions into language-specific code.
- CORBA IDL has a rich type system, including
  - basic types (e.g., long, short, float, char, boolean),
  - constructed types (e.g., struct, union, sequence, array, enum), and objects.
- Parameter passing modes in CORBA IDL include **in**, **out**, **inout**
  - It supports basic & constructed types passed by value and objects passed by reference

Examples of OOM

CORBA Services

CORBA provides a range of services to support distributed computing:

- Naming Service: Maps names to remote object references.
- Trading Service: Maps attributes (properties) to remote object references.
- Persistent Object Service: Enables the implementation of persistent CORBA objects.
- Transaction Service: Allows object invocations to be part of transactions.
- Event Service and Notification Service: Support asynchronous communication, built on top of synchronous communication with push or pull options.
    - Note that this is not an integrated programming model with general IDL messages

Examples of OOM

Advantages and Disadvantages CORBA

Advantages

- Natural OOP paradigm
- Location transparency
- Multi-language support (CORBA)

Disadvantages

- The primary communication pattern in traditional OOM is synchronous request/reply, although
  - CORBA added "oneway" semantics and asynchronous Method Invocation (AMI) to address this limitation.
  - However, implementations may not always be loosely coupled.
- Distributed garbage collection
  - (releasing memory for unused remote objects) is a complex challenge in OOM systems.
- OOM systems tend to be rather static and heavy-weight,
  - which can make them less suitable for ubiquitous computing and embedded devices.

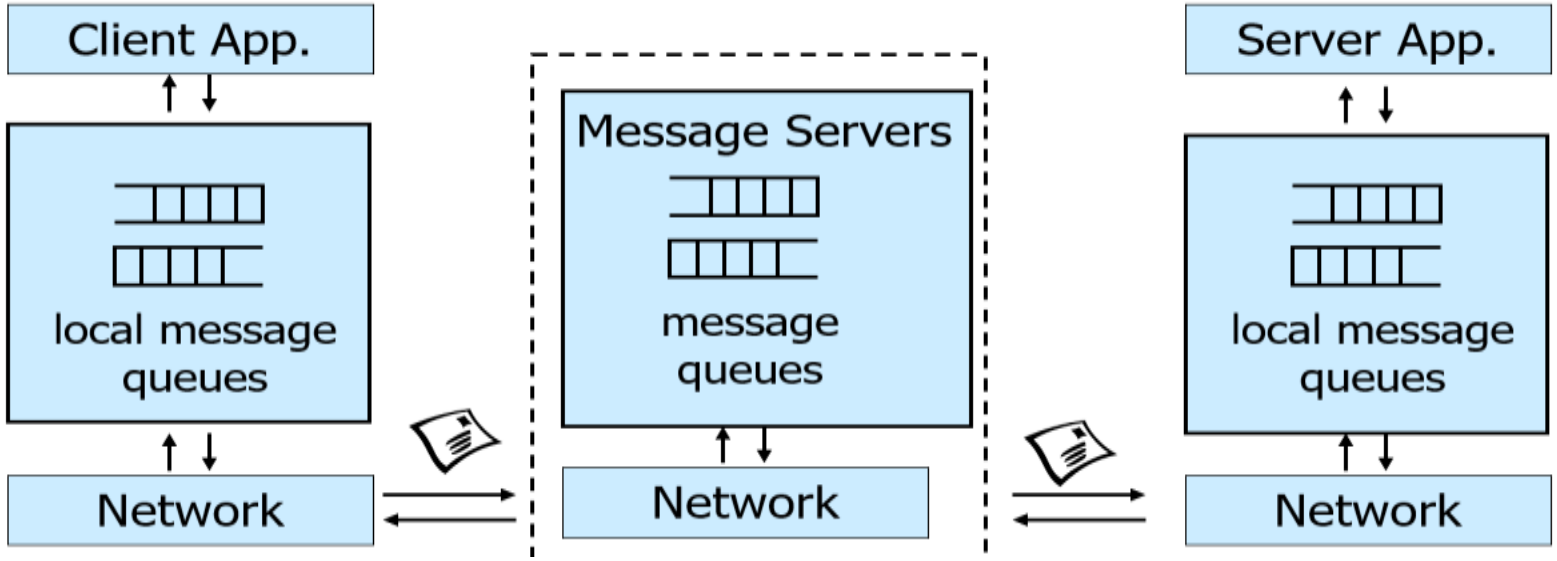
Examples of OOM

Reflective Middleware

- Reflective Middleware is presented as a more flexible form of OOM, designed for mobile and context-aware applications.
- It enables adaptation to context by monitoring and dynamically substituting components.
- Reflective Middleware provides:
  - Interfaces for reflection
    - allowing objects to inspect middleware behavior, and
  - Interfaces for customizability,
    - Modify middleware behavior
- Enabling dynamic reconfiguration based on the environment (different protocols or QoS levels).
- An example is using different marshalling strategies over unreliable wireless links.

Message-Oriented Middleware (MOM)

- Communication using **messages,** where messages are the fundamental unit of data exchange.
- Messages stored in **message queues**
- **Message servers** decouple client and server, providing asynchronous communication
- Various assumptions about **message content**



Message-Oriented Middleware (MOM)

MOM Properties

- MOM is characterized by **Asynchronous** interaction
  - Client and server are only **loosely coupled**
  - Messages are queued
  - Good for application integration
- MOM Support for **reliable** delivery service
  - Keep queues in persistent storage
- Processing of messages by intermediate message server(s)
  - May do filtering, transforming, logging, …
- MOM organized as Networks of message servers, which is well-suited for database integration.

Message-Oriented Middleware (MOM)

Examples of MOM

IBM MQSeries:

- IBM MQSeries provides one-to-one reliable message passing using queues.
  - It supports both persistent and non-persistent messages
  - Message priorities, and message notification.
- Key components include
  - Queue Managers: Responsible for queues and message transfer
    - Keep routing tables
  - Message Channels: reliable connections between queue managers.
- The MQSeries API includes functions like

MQopen	Open a queue
MQclose	Close a queue
MQput	Put message into opened queue
MQget	Get message from local queue

Message-Oriented Middleware (MOM)

Examples of MOM

Java Message Service (JMS):

- JMS is a Java API specification for accessing MOM implementations.
- It defines two main modes of operation:
  - Point-to-point: One-to-one communication using queues.
  - Publish/Subscribe: A one-to-many communication model (similar to Event-Based Middleware).
- A JMS Server implements the JMS API, and JMS Clients connect to JMS servers.
- Java objects can be serialized into JMS messages, and a JMS interface is available for MQ.

Message-Oriented Middleware (MOM)

Advantages and Disadvantages

Advantages

- Loose coupling
- Reliable delivery
- Good for application integration

Disadvantages

- MOM can have a poor programming abstraction (although it has evolved) and is considered rather low-level.
- Implementing request/reply patterns can be more complex in MOM, although it is possible.
- Message formats were originally unknown to the middleware, leading to a lack of type checking (JMS addresses this to some extent).
- The queue abstraction in basic MOM provides only one-to-one communication, which can limit scalability (JMS publish/subscribe addresses this)

Web Services

- Use well-known web standards for distributed computing

Communication

- Message content expressed in **XML(**Extensible Markup Language)
- **S**imple **O**bject **A**ccess **P**rotocol (**SOAP**)
  - Lightweight protocol for sync/async communication

Service Description

- **W**eb **S**ervices **D**escription **L**anguage (**WSDL**)
  - used to define the interface of a web service

Service Discovery

- **U**niversal **D**escription **D**iscovery and **I**ntegration (**UDDI**)
  - provides a directory for discovering web services and their WSDL descriptions

Web Services

Web Services Properties

- Web services are designed to be language-independent and based on open standards.
- **SOAP** offers OOM and MOM-style communication:
  - Synchronous request/reply like OOM
  - Asynchronous messaging like MOM
  - Supports internet transports (http, smtp, ...)
  - Uses XML Schema for marshalling types between programming languages & XML representations
- **WSDL** says how to use a web service **http://api.google.com/GoogleSearch.wsdl**
- **UDDI** helps to find the right web service
  - Exports SOAP API for access

Web Services

Advantages and Disadvantages

Advantages

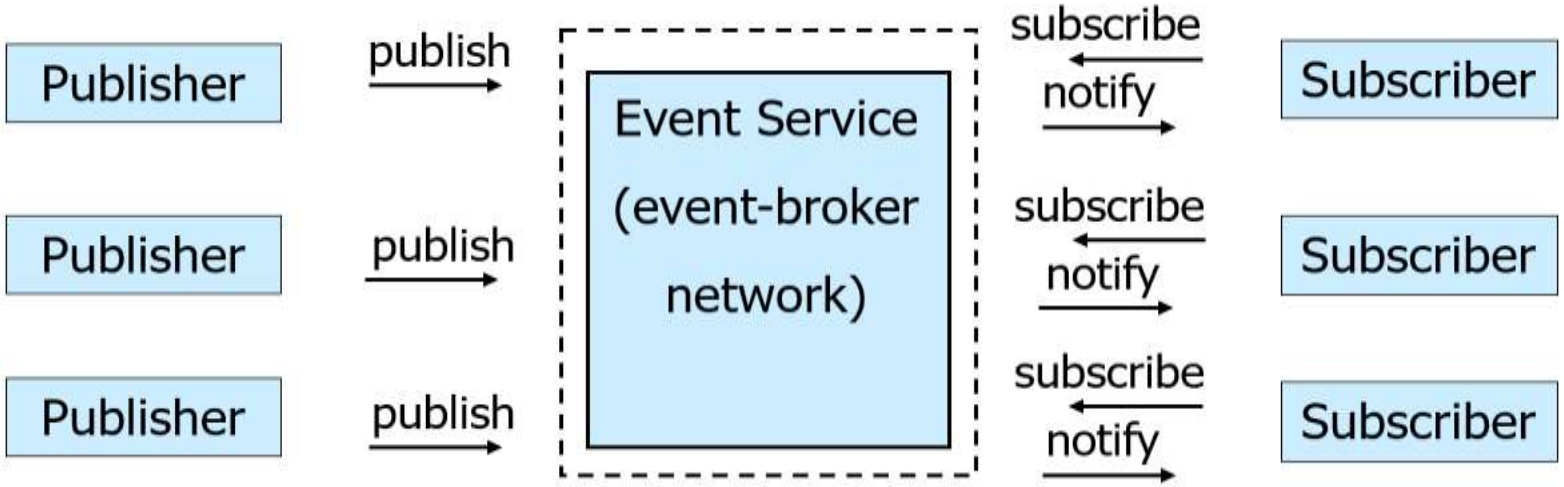
- Language/platform-independent.
- Uses existing web infrastructure (HTTP, SMTP).

Disadvantages

- Web services can be considered a low-level abstraction, requiring developers to implement many interaction patterns themselves.
- While request/reply is provided, other patterns (like one-to-many or notification) need to be built on top of the basic mechanisms.
- Service invocation is typically synchronous, rather than notification-based.
- No nested/grouped invocations or transactions.
- Do not provide location transparency.

Event-Based Middleware (Publish/Subscribe)

- In Event-Based Middleware (also known as Publish/Subscribe)
- Publishers *advertise* or publish events (messages)
- Subscribers express their interest in specific events by creating subscriptions.
- An Event Service (or event broker network) is responsible for notifying interested subscribers when relevant events are published.
- Events can contain arbitrary content (with typing) or be structured as name/value pairs.



Event-Based Middleware (Publish/Subscribe)

Topic-Based and Content-Based Publish/Subscribe

- The Event Service matches events against subscriptions to determine which subscribers should be notified.
- Two main types of Publish/Subscribe are:

Topic-Based Publish/Subscribe

- Publishers publish events under a specific topic or subject
- Subscribers subscribe to a **topic**

E.g. subscribe(PrintJobFinishedTopic, …)

(Topic and) Content-Based Publish/Subscribe

- Publishers publish events belonging to **topic**s and
- Subscribers can **filter** events based on their content

E.g. subscribe(type=printjobfinished, printer='aspen', …)

Event-Based Middleware (Publish/Subscribe)

Properties of Publish/Subscribe

- Publish/Subscribe enables asynchronous communication
  - loose coupling between publishers and subscribers.
- It supports many-to-many interaction,
  - Making it suitable for large-scale systems.
  - Publishers do not need to know about subscribers, and vice versa
  - Allowing for dynamic joining and leaving of participants (publishers, subscribers, and brokers).
- Content-based Publish/Subscribe is highly expressive
  - Enabling filtering and delivery of information only to interested parties.
  - Efficient content-based routing can be achieved through broker networks.

Event-Based Middleware (Publish/Subscribe)

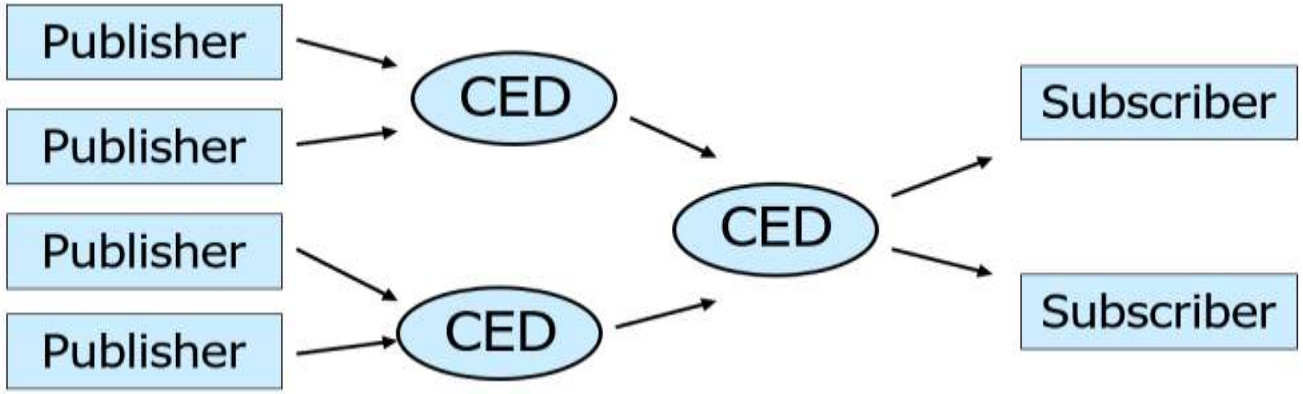
Composite Event Detection (CED)

- Content-based Publish/Subscribe may not be sufficient for complex scenarios with many event types.
- CED addresses this by allowing subscribers to express interest in event patterns (defining high-level events).
- Composite Event Detectors subscribe to primitive events and publish composite events, which represent more complex situations or relationships between events.

Event Patterns E.g.

PrinterOutOfPaperEvent or

PrinterOutOfTonerEvent



Event-Based Middleware (Publish/Subscribe)

Advantages and Disadvantages

Advantages

- Fully decoupled
- Many-to-many communication
- Highly scalable
- Dynamic membership

Disadvantages

- No guaranteed delivery
- Complex filtering overhead
- Potential event flooding

Choosing the Right Middleware

Decision Guide

Requirement	Recommended Middleware
Simple client-server	RPC
Object-oriented system	OOM (RMI/CORBA)
Reliable enterprise integration	MO
Real-time data distribution	Pub/Sub
Web/cloud integration	Web Services

Understanding these middleware concepts gives you the foundation to:

1. Design better distributed systems
2. Troubleshoot complex issues
3. Choose the right technologies for your needs

Middleware Architectures and Technologies

Comparative Summary

Feature	*RPC*	*OOM*	*MOM*	*Pub/Sub*	*Web Services*
Communication Style	Synchronous (Request-Reply)	Synchronous (Method Invocation)	Asynchronous (Message Queuing)	Asynchronous (Event Notification)	Both (Request-Reply & Messaging)
Interaction Pattern	(1:1)	(1:1)	(1:1)	(1:N)	(1:1) & (1:N)
Communication Mechanism	Request and wait for a response	Invoke methods on remote objects	Send and receive messages via queues	Publish events; subscribers receive notifications	Exchange data over a network (often HTTP)
Coupling	Tight (Direct interaction)	Tight (Direct object interaction)	Loose (Indirect via queues)	Very Loose (Decoupled by events)	Loose (Standards-based interaction)
Scalability	Moderate	Moderate	High	Very High	High
Typical Protocols	Custom binary protocols	IIOP (CORBA), JRMP (Java RMI)	AMQP, JMS	Custom protocols	HTTP + XML/JSON (SOAP, REST)
Error Handling	Client retries, error codes	Exceptions in method calls	Guaranteed message delivery	Best-effort delivery	HTTP status codes, application-specific error handling
Use Case	Internal communication within a distributed system	Building distributed applications with objects	Integrating applications, reliable messaging	Real-time event distribution, notifications	Interoperability over the web
Examples	Internal service calls in a microservices architecture	Distributed object systems, older enterprise middleware	Order processing, financial transactions, asynchronous tasks	Real-time data feeds (stock quotes), chat systems, news distribution	APIs for e-commerce, cloud services, web-based applications

Thanks !